

Algoritmi e Strutture Dati

Introduzione

Alberto Montresor

Università di Trento

2026/09/15

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

- 1 Introduzione
- 2 Problemi e algoritmi
 - Primi esempi
 - Pseudo-codice
- 3 Valutazione
 - Efficienza
 - Correttezza
- 4 Conclusioni

Introduzione

Problema computazionale

Dati un **dominio di input** e un **dominio di output**, un *problema computazionale* è rappresentato dalla **relazione matematica** che associa ogni elemento del dominio di input ad uno o più elementi del dominio di output.

Algoritmo

Dato un problema computazionale, un *algoritmo* è un procedimento **effettivo**, espresso tramite un insieme di **passi elementari ben specificati** in un sistema **formale** di calcolo, che risolve il problema in tempo **finito**.

Un po' di storia

- Papiro di Rhind o di Ahmes (1850BC): algoritmo del contadino per la moltiplicazione
- Algoritmi di tipo numerico furono studiati da matematici babilonesi ed indiani
- Algoritmi in uso fino a tempi recenti furono studiati dai matematici greci più di 2000 anni fa
 - Algoritmo di Euclide per il massimo comune divisore
 - Algoritmi geometrici (calcolo di tangenti, sezioni di angoli, ...)



https://en.wikipedia.org/wiki/Rhind_Mathematical_Papyrus

Origine del nome

Abu Abdullah Muhammad bin Musa **al-Khwarizmi**

- È stato un matematico, astronomo, astrologo e geografo
- Nato in Uzbekistan, ha lavorato a Baghdad
- Dal suo nome: **algoritmo**



Algoritmi de numero indorum

- Traduzione latina di un testo arabo ormai perso
- Ha introdotto i numeri indiani (arabi) nel mondo occidentale
- Dal numero arabo **sifr** = 0: zephirum → zevero → zero, ma anche cifra



https://en.wikipedia.org/wiki/Muhammad_ibn_Musa_al-Khwarizmi

Origine del nome

Abu Abdullah Muhammad bin Musa **al-Khwarizmi**

- È stato un matematico, astronomo, astrologo e geografo
- Nato in Uzbekistan, ha lavorato a Baghdad
- Dal suo nome: **algoritmo**

Al-Kitab al-muhtasar fi hisab **al-gabr** wa-l-muqabala

- La sua opera più famosa (820 d.C.)
- Tradotta in latino con il titolo:
Liber algebrae et almucabala
- Dal suo titolo: **algebra**



https://en.wikipedia.org/wiki/Muhammad_ibn_Musa_al-Khwarizmi

Problemi computazionali: esempi

Esempio: Minimo

Il **minimo di un insieme** S è l'elemento di S che è minore o uguale ad ogni elemento di S .

$$\min(S) = a \Leftrightarrow \exists a \in S : \forall b \in S : a \leq b$$

Esempio: Ricerca

Sia $A = a_0, a_1, \dots, a_{n-1}$ una sequenza di dati ordinati e distinti, i.e. $a_0 < a_1 < \dots < a_{n-1}$. **Ricerca la posizione di un dato** v in S consiste nel restituire un indice i tale che $0 \leq i \leq n - 1$, se v è presente nella posizione i , oppure -1 , se v non è presente.

$$\text{lookup}(A, v) = \begin{cases} i & \exists i \in \{0, \dots, n - 1\} : a_i = v \\ -1 & \text{altrimenti} \end{cases}$$

Algoritmi: esempi

Algoritmo: Minimo

Per trovare il minimo di un insieme, confronta ogni elemento con tutti gli altri; l'elemento che è minore di tutti è il minimo.

Algoritmo: Ricerca

Per trovare un valore v nella sequenza S , confronta v con tutti gli elementi di S , in sequenza, e restituisci la posizione corrispondente; restituisci -1 se nessuno degli elementi corrisponde.

Problemi

Le descrizioni precedenti presentano diversi problemi:

- **Descrizione**

- Descritti in linguaggio naturale, imprecisi
- Abbiamo bisogno di un linguaggio più formale

- **Valutazione**

- Esistono algoritmi “migliori” di quelli proposti?
- Dobbiamo definire il concetto di migliore

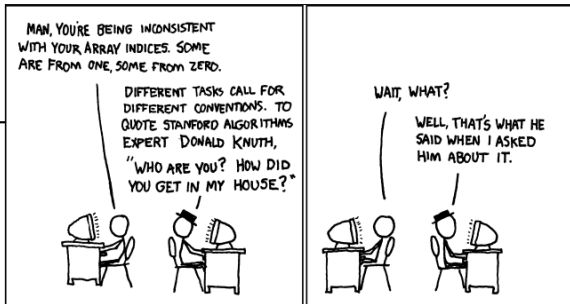
Come descrivere un algoritmo

- È necessario utilizzare una descrizione il più possibile formale
- Indipendente dal linguaggio: “Pseudo-codice”
- Particolare attenzione va dedicata al livello di dettaglio
 - Da una ricetta di canederli, leggo:
“... amalgamate il tutto e fate riposare un quarto d'ora...”
 - Cosa significa “amalgamare”? Cosa significa “far riposare”?
 - E perché non c'è scritto più semplicemente “prepara i canederli”?

Esempio: pseudo-codice

```
int min(int[] S, int n)
for i = 0 to n - 1 do
    boolean isMin = true
    for j = 0 to n - 1 do
        if i ≠ j and S[j] < S[i]
            then
                isMin = false
    if isMin then
        return S[i]
```

```
int lookup(int[] S, int n, int v)
for i = 0 to n - 1 do
    if S[i] == v then
        return i
return -1
```



Pseudo-codice

- $a = b$, $a += b$, $a -= b$
- $a, b = b, a \equiv$
 $tmp = a; a = b; b = tmp$
- $T[]$ $A = \mathbf{new} T[0 \dots n - 1] = \{0\}$
- $T[][]$ $B =$
 $\mathbf{new} T[0 \dots n - 1][0 \dots m - 1]$
- **int**, **real**, **boolean**, **int**
- **and**, **or**, **not**
- $==$, $\neq$, $\leq$, $\geq$
- $+$, $-$, $\cdot$, $/$, $\lfloor x \rfloor$, $\lceil x \rceil$, $\log$, x^2 , ...
- **iif**(*condizione*, v_1 , v_2)
- **if** *condizione* **then** istruzione
- **if** *condizione* **then** istruzione1 **else** istruzione2
- **while** *condizione* **do** istruzione
- **foreach** *elemento* $\in$ *insieme* **do** istruzione
- **return**
- *% commento*

Pseudo-codice

- **for** *indice* = *estremoInf* **to** *estremoSup* **do** istruzione
 int *indice* = *estremoInf*
 while *indice* $\leq$ *estremoSup* **do**
 | *istruzione*
 | *indice* = *indice* + 1
- **for** *indice* = *estremoSup* **downto** *estremoInf* **do** istruzione
 int *indice* = *estremoSup*
 while *indice* $\geq$ *estremoInf* **do**
 | *istruzione*
 | *indice* = *indice* - 1
- RECTANGLE *r* = **new** RECTANGLE
- *r.altezza* = 10
- **delete** *r*
- *r* = **nil**

 RECTANGLE

int *lunghezza*
int *altezza*

Come valutare l'algoritmo

Risolve il problema in modo **efficiente**?

- Dobbiamo stabilire come valutare se un programma è efficiente
- Alcuni problemi non possono essere risolti in modo efficiente
- Esistono soluzioni “ottime”: non è possibile essere più efficienti

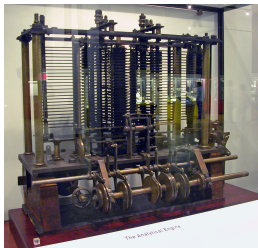
Risolve il problema in modo **corretto**?

- Dimostrazione matematica, descrizione “informale”
- Nota: Alcuni problemi non possono essere risolti
- Nota: Alcuni problemi vengono risolti in modo approssimato

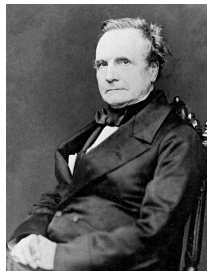
Charles Babbage

Passages from the Life of a Philosopher, Charles Babbage, 1864

As soon as an Analytical Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will then arise — **By what course of calculation can these results be arrived at by the machine in the shortest time?**



Modello della macchina analitica, Museo della Scienza Londra, foto Bruno Barral



Charles Babbage, 1860

Valutazione algoritmi – Efficienza

Complessità di un algoritmo

Analisi delle **risorse** impiegate da un algoritmo per risolvere un problema, in funzione della **dimensione** e dalla **tipologia** dell'input

Risorse

- **Tempo**: tempo impiegato per completare l'algoritmo
 - Misurato con il cronometro?
 - Misurato contando il numero di operazioni rilevanti?
 - Misurato contando il numero di operazioni elementari?
- **Spazio**: quantità di memoria utilizzata
- **Banda**: quantità di bit spediti (algoritmi distribuiti)

Definizione di tempo

Tempo $\equiv$ wall-clock time

Il tempo effettivamente impiegato per eseguire un algoritmo

Dipende da troppi parametri:

- bravura del programmatore
- linguaggio di programmazione utilizzato
- codice generato dal compilatore
- processore, memoria (cache, primaria, secondaria)
- sistema operativo, processi attualmente in esecuzione

Dobbiamo considerare una rappresentazione più astratta!

Definizione di tempo – A grandi linee

Tempo $\equiv$ n. operazioni rilevanti

Numero di operazioni "rilevanti", ovvero il numero di operazioni che caratterizzano lo scopo dell'algoritmo.

Esempio

- Nel caso del minimo, numero di confronti $<$
- Nel caso della ricerca, numero di confronti $==$

Proviamo!

Valutazione algoritmi – Minimo

Contiamo il numero di confronti per il problema del **minimo**

```
int min(int[] S, int n)
for i = 0 to n - 1 do
    boolean isMin = true
    for j = 0 to n - 1 do
        if i ≠ j and S[j] < S[i] then
            isMin = false
    if isMin then
        return S[i]
```

Algoritmo “naïf”: $n^2 - n$

Si può fare meglio di così?

Valutazione algoritmi – Un algoritmo migliore

Contiamo il numero di confronti per il problema del **minimo**

```
int min(int[] A, int n)
int minSoFar = A[0]
for i = 1 to n - 1 do
    if A[i] < minSoFar then
        minSoFar = A[i]
return minSoFar
```

Algoritmo “naïf”: $n^2 - n$

Algoritmo efficiente: $n - 1$

Valutazione algoritmi – Ricerca

Contiamo il numero di confronti per il problema della **ricerca**

```
int lookup(int[ ] A, int n, int v)
```

```
for  $i = 0$  to  $n - 1$  do
```

```
    if  $A[i] == v$  then  
        return  $i$ 
```

```
return  $-1$ 
```

Algoritmo “naïf”: n

Si può fare meglio di così?

Valutazione algoritmi – Un algoritmo migliore

Una soluzione più efficiente

Analizzo l'elemento centrale (indice m) del sottovettore considerato:

- Se $A[m] = v$, ho trovato il valore cercato
- Se $v < A[m]$, cerco nella “metà di sinistra”
- Se $A[m] < v$, cerco nella “metà di destra”

Ricerca del numero 50:

1	3	7	8	12	16	21	34	37	45	50	67	68	72	88
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

Valutazione algoritmi – Un algoritmo migliore

Una soluzione più efficiente

Analizzo l'elemento centrale (indice m) del sottovettore considerato:

- Se $A[m] = v$, ho trovato il valore cercato
- Se $v < A[m]$, cerco nella “metà di sinistra”
- Se $A[m] < v$, cerco nella “metà di destra”

Ricerca del numero 50:

1	3	7	8	12	16	21	34	37	45	50	67	68	72	88
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

m

Valutazione algoritmi – Un algoritmo migliore

Una soluzione più efficiente

Analizzo l'elemento centrale (indice m) del sottovettore considerato:

- Se $A[m] = v$, ho trovato il valore cercato
- Se $v < A[m]$, cerco nella “metà di sinistra”
- Se $A[m] < v$, cerco nella “metà di destra”

Ricerca del numero 50:

1	3	7	8	12	16	21	34	37	45	50	67	68	72	88
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

m

Valutazione algoritmi – Un algoritmo migliore

Una soluzione più efficiente

Analizzo l'elemento centrale (indice m) del sottovettore considerato:

- Se $A[m] = v$, ho trovato il valore cercato
- Se $v < A[m]$, cerco nella “metà di sinistra”
- Se $A[m] < v$, cerco nella “metà di destra”

Ricerca del numero 50:

1	3	7	8	12	16	21	34	37	45	50	67	68	72	88
									m					

Valutazione algoritmi – Un algoritmo migliore

Una soluzione più efficiente

Analizzo l'elemento centrale (indice m) del sottovettore considerato:

- Se $A[m] = v$, ho trovato il valore cercato
- Se $v < A[m]$, cerco nella “metà di sinistra”
- Se $A[m] < v$, cerco nella “metà di destra”

Ricerca del numero 50:

1	3	7	8	12	16	21	34	37	45	50	67	68	72	88
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

m

Valutazione algoritmi – Un algoritmo migliore

Contiamo il numero di confronti per il problema della **ricerca**

```
int bsRec(int[] A, int v, int start, int end)
if start > end then
    return -1                % Vettore vuoto
else
    int m = ⌊(start + end)/2⌋
    if A[m] == v then
        return m             % Trovato
    else if A[m] < v then
        return bsRec(A, v, m + 1, end)
        % Sottovettore di destra
    else
        return bsRec(A, v, start, m - 1)
        % Sottovettore di sinistra
```

Algoritmo “naïf”: n

Algoritmo efficiente:

$2\lceil \log n \rceil$

Un po' di storia

- 1817: Metodo della bisezione per trovare le radici di una funzione (Bolzano)
- 1946: Prima menzione di binary search (John Mauchly, progettista di ENIAC)
- 1960: Prima versione di binary search che lavora con vettori di dimensione arbitraria (!) (Derrick Henry Lehmer)

Although the basic idea of binary search is comparatively straightforward, the details can be surprisingly tricky.

Donald Knuth, The Art of Computer Programming

<https://hsm.stackexchange.com/questions/2200/what-is-the-first-historical-reference-to-the-binary-search-algorithm>
<https://devopedia.org/binary-search>

Problemi di overflow

```
int bsRec(int[] A, int v, int start, int end)
if start > end then
    return -1                % Vettore vuoto
else
    int m = ⌊start + (end - start)/2⌋
    if A[m] == v then
        return m            % Trovato
    else if A[m] < v then
        return bsRec(A, v, m + 1, end)
        % Sottovettore di destra
    else
        return bsRec(A, v, start, m - 1)
        % Sottovettore di sinistra
```

Algoritmo efficiente:

$$2^{\lceil \log n \rceil}$$

Valutazione algoritmi – Correttezza

Invariante

Condizione sempre vera in un certo punto del programma

Invariante di ciclo

- Una condizione sempre vera all'inizio dell'iterazione di un ciclo
- Cosa si intende per "inizio dell'iterazione"?

Invariante di classe

- Una condizione sempre vera al termine dell'esecuzione di un metodo della classe

Valutazione algoritmi – Correttezza

Il concetto di **invariante di ciclo** ci aiuta a dimostrare la correttezza di un **algoritmo iterativo**.

- **Inizializzazione** (caso base):
La condizione è vera alla prima iterazione di un ciclo
- **Conservazione** (passo induttivo):
Se la condizione è vera prima di un'iterazione del ciclo, allora rimane vera al termine (quindi prima della successiva iterazione)
- **Conclusione**:
Quando il ciclo termina, l'invariante deve rappresentare la "correttezza" dell'algoritmo

Valutazione algoritmi – Correttezza

Invariante

All'inizio di ogni iterazione del ciclo **for**, la variabile *minSoFar* contiene il minimo parziale degli elementi $A[0 \dots i - 1]$.

```
int min(int[] A, int n)
int minSoFar = A[0]
for i = 1 to n - 1 do
    if A[i] < minSoFar then
        minSoFar = A[i]
return minSoFar
```

Inizializzazione

Conservazione

Conclusione

Valutazione algoritmi – Correttezza

La dimostrazione per induzione è utile anche per gli algoritmi ricorsivi

```
int bsRec(int[] A, int v, int start, int end)
if start > end then
    return -1                % Vettore vuoto
else
    int m = ⌊(start + end)/2⌋
    if A[m] == v then
        return m             % Trovato
    else if A[m] < v then
        return bsRec(A, v, m + 1, end)
        % Sottovettore di destra
    else
        return bsRec(A, v, start, m - 1)
        % Sottovettore di sinistra
```

Per induzione sulla
dimensione n dell'input

- **Caso base:**
 $n = 0$ ($start > end$)
- **Ipotesi induttiva:**
vero per tutti gli $n' < n$
- **Passo induttivo:**
dimostrare che è vero
per n

Altre proprietà

Semplicità, modularità, manutenibilità, espandibilità, robustezza, ...

- Secondari in un corso di algoritmi e strutture dati
- Fondamentali per un corso di ingegneria del software

Commento

Alcune proprietà hanno un costo aggiuntivo in termini di prestazioni

- Codice modulare → costo gestione chiamate
- Java bytecode → costo interpretazione

Progettare algoritmi efficienti è un prerequisito per poter pagare questo costo